MINI PROJECT REPORT

Predicting Bazel Build Times Using Machine Learning

Reproduction of the Reference Results and Extension with Random Forest Regression

Reference paper: Muhammad Umar Nadeem and Sarah Raza
Reference title: “Predicting Bazel Build Times Using Machine Learning”

Abstract

Objective. This project reproduces the machine-learning pipeline described in the reference paper for predicting the CPU time of Bazel builds from commit-level file-path and file-type information, and then extends the work with one additional regression model. The reference uses linear regression with categorical preprocessing and an optional prefix x type feature crossing. The project adds a Random Forest Regressor as the extension model.

Dataset and evaluation. The supplied processed dataset contains 500 Bazel commit examples. The experiments use 300 training, 100 validation, and 100 testing examples and evaluate performance with root mean squared error (RMSE). The paper reports average training RMSE values of 5735.84 ms without feature crossing and 5628.09 ms with feature crossing.

Results. The independent reproduction obtained average training RMSE values of 5707.02 ms without feature crossing and 5685.62 ms with feature crossing. These differ from the corresponding reported values by approximately 0.50% and 1.02%, respectively. On the common 100-example test split at the final training period, the reproduced linear models achieved 5796.58 ms and 5801.74 ms RMSE, while Random Forest achieved 5881.74 ms. Thus, in this experiment, Random Forest performed worse on the shared test split.

Contents

- 1. Introduction and Project Objectives
- 2. Reference Study and Problem Definition
- 3. Dataset and Preprocessing
- 4. Part 1 - Reproduction of the Reported Results
- 5. Part 2 - Additional Model: Random Forest Regressor
- 6. Comparative Analysis and Discussion
- 7. Instructor Requirement Traceability
- 8. Limitations and Reproducibility Notes
- 9. Conclusion
- 10. References

1. Introduction and Project Objectives

Build systems such as Bazel are used to automate software compilation and testing. The reference study investigates whether commit-level information can be used to predict the CPU time required for a Bazel build. The paper focuses on two categorical descriptors of changed files: the most common file-path prefix and the most common file type. It then applies linear regression, with a feature-crossing variant, to predict CPU time in milliseconds.

This report follows the instructor requirement in two stages. First, it independently reconstructs the pipeline described in the reference rather than copying the authors' Python implementation. Second, it adds one model not used in the reference, trains it on the same processed representation and split, and evaluates it with the same RMSE metric.

Project objectives

- Reproduce the reference preprocessing, model family, hyperparameters, split structure, and evaluation metric as closely as the paper permits.
- Compare the reproduced results with the numerical results explicitly reported in the paper.
- Explain reasonable sources of discrepancy where the reproduced values do not exactly match the paper.
- Extend the experiment with one additional model using the same dataset, split, feature representation, and RMSE metric.
- Report the comparative findings without claiming that the additional model must outperform the reference model.

2. Reference Study and Problem Definition

The assigned reference is "Predicting Bazel Build Times Using Machine Learning" by Muhammad Umar Nadeem and Sarah Raza. The paper states that its dataset is built from 500 commits of the open-source bazelbuild/bazel project. For each commit, the authors use the most common prefix of changed file paths and the most common file type as predictors, while CPUTime from Bazel Build Event Protocol data is used as the response variable.

The paper describes linear regression as the main learning method. It then introduces feature crossing, where the prefix and file-type categories are combined so that the model can distinguish specific prefix/type combinations. The paper reports that this feature crossing lowers average training RMSE relative to the non-crossed linear model.

Component	Reference specification
Problem	Regression: predict Bazel build CPU time
Dataset	500 commit/build examples from bazelbuild/bazel
Input features	Most common file-path prefix; most common file type
Target	CPUTime in milliseconds
Reference model	Linear Regression trained with stochastic gradient descent
Feature variant	Optional prefix x type feature crossing
Reported learning rate	0.01
Reported batch size	16
Data split described in paper	300 training, 100 validation, 100 testing

Metric	RMSE
Training periods shown in paper	9

3. Dataset and Preprocessing

Two processed files supplied for this reproduction were used: InputData.csv and CPUTimes.csv. InputData.csv contains 500 rows with the fields ID, prefix, and type. CPUTimes.csv contains 500 rows with commitID and CPUTime. The supplied target identifiers run from 1 through 500, and the input and target files do not share a common commit key. Therefore, the report's implementation aligns the two supplied files by row order, which is the reproducible linkage available from the supplied data.

Dataset property	**Observed value**
Input records	500
Target records	500
Training / validation / test	300 / 100 / 100
Observed prefix categories in supplied data	8
Observed file types in supplied data	4
CPU time minimum	564 ms
CPU time median	1837.5 ms
CPU time mean	4299.40 ms
CPU time maximum	44863 ms

Feature vocabulary. The paper lists the prefix vocabulary as bazelci/, examples/, scripts/, site/, src/conditions, src/java_tools, src/main, src/test, src/tools, third_party/, and tools/. The listed type vocabulary is JAVA, C/C++, Starlark, python, and HTML/CSS/JS. The supplied dataset uses a subset of those categories.

Independent encoding. The paper describes converting categorical strings into discrete/bucketized representations. For the independent implementation, those fixed categorical vocabularies were represented as one-hot vectors. For the feature-crossing experiment, an additional one-hot block represents each observed prefix/type combination. This preserves the categorical information required by the linear model while keeping the implementation independent of the authors' Python source.

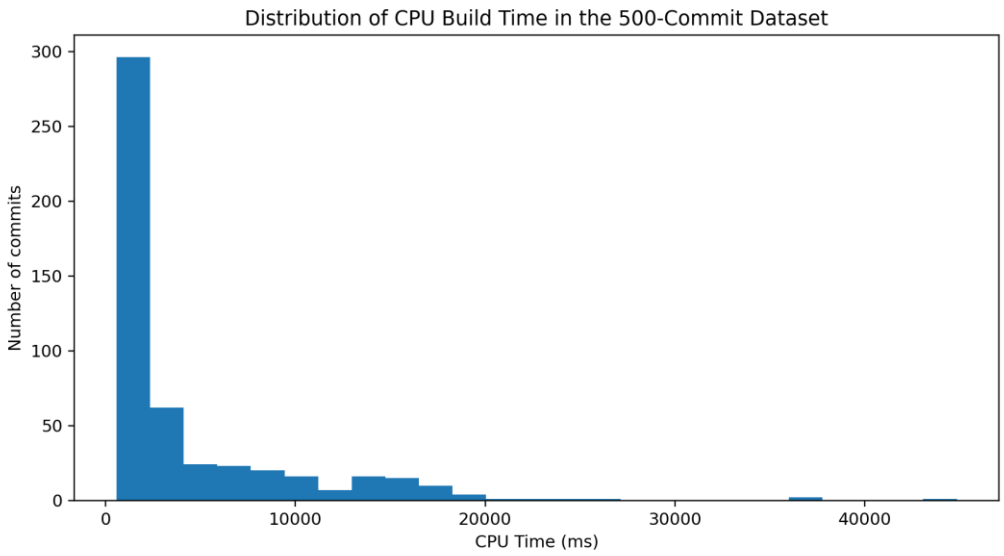


Figure 1. CPU-time distribution for the supplied 500-commit dataset.

4. Part 1 - Reproduction of the Reported Results

4.1 Reproduction methodology

The Part 1 implementation was reconstructed from the paper and the supplied CSV files. The authors' Python implementation was not used as the basis of the code. The reproduced procedure is: load the 500 aligned examples; encode prefix and type categorically; split the examples into the documented 300/100/100 groups in the supplied row order because the paper does not specify a shuffle or random seed; train a linear regression model with mini-batch stochastic gradient descent using learning rate 0.01 and batch size 16; record RMSE over the 9 periods shown in the paper; then repeat the experiment with an added prefix x type categorical feature-crossing block.

Setting	Reproduction choice	Reason
Split	300 train / 100 validation / 100 test, row order	Paper gives sizes but not a split seed or shuffling rule
Learning rate	0.01	Explicitly reported in the paper
Batch size	16	Explicitly reported in the paper
Training periods	9	Nine result periods are shown in the paper
Optimizer	Mini-batch stochastic gradient descent	Paper states SGD for the linear model
Feature crossing	Prefix x type categorical cross	Explicitly described as the second experiment
Metric	RMSE	Primary metric reported by the paper

Paper result used for comparison. The paper explicitly reports average training RMSE values of 5735.84 ms for linear regression without feature crossing and 5628.09 ms with feature crossing. The paper's displayed nine-period values also average to approximately 5628.31 ms for the feature-crossed case, so the reported text value of 5628.09 ms is retained as the official reference value in this report.

Experiment	Paper reported training RMSE (ms)	Reproduced training RMSE (ms)	Difference
Linear Regression - without feature crossing	5735.84	5707.02	-0.50%
Linear Regression - with prefix x type crossing	5628.09	5685.62	+1.02%

Interpretation. Both reproduced training RMSE values are close to the reported values. The non-crossed model is approximately 28.82 ms below the paper's reported value, while the crossed model is approximately 57.54 ms above it. These differences are small relative to RMSE values around 5.6-5.7 seconds and are consistent with the instructor's statement that small differences are acceptable.

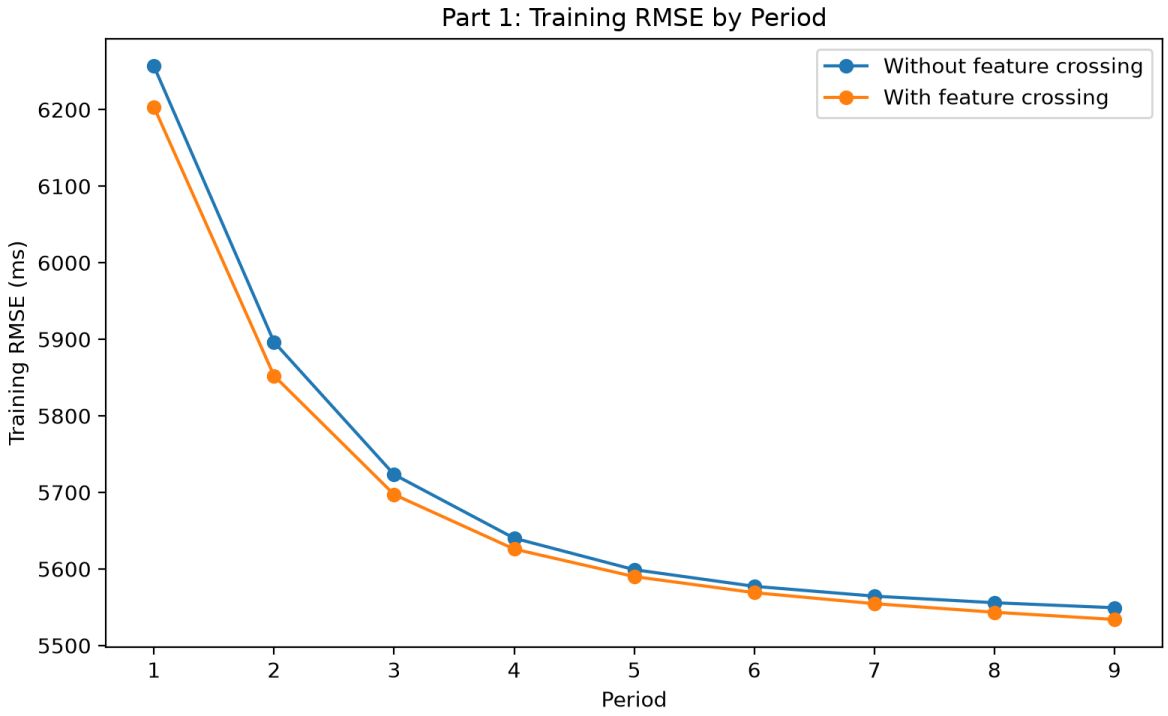


Figure 2. Training RMSE across the 9 reproduction periods.

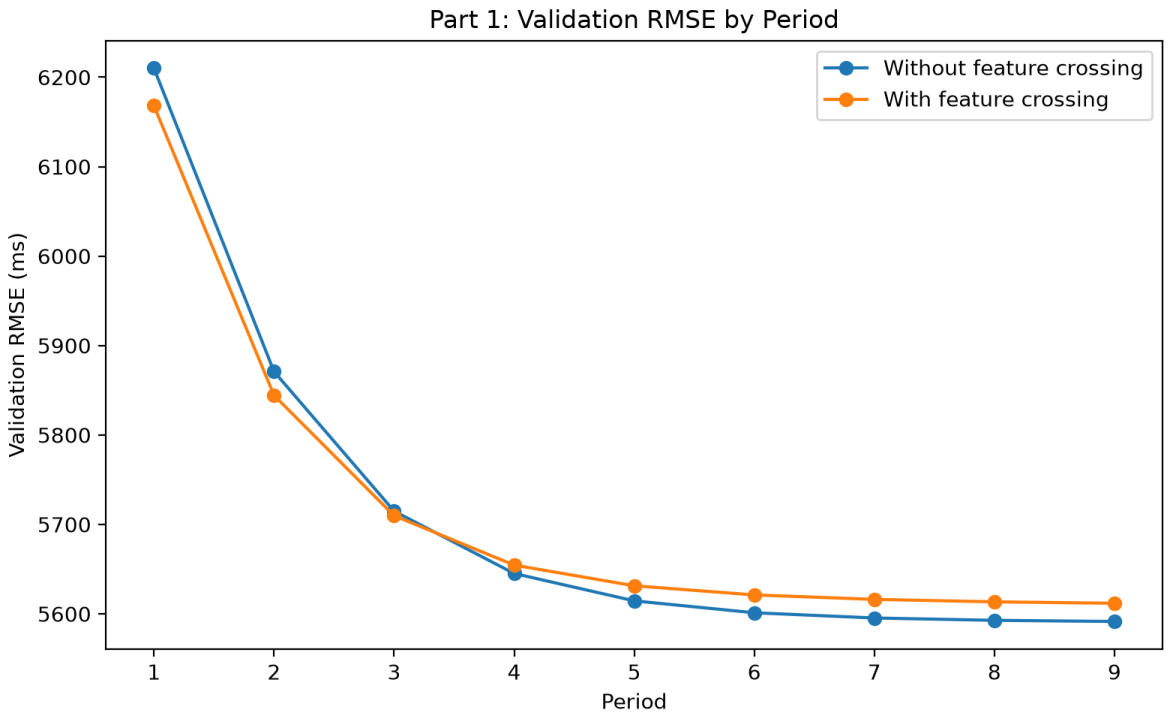


Figure 3. Validation RMSE across the 9 reproduction periods.

Learning behavior. Both linear-regression variants show a strong decline in RMSE during the first few periods and then a gradual flattening. The feature-crossed model has lower training RMSE than the non-crossed model throughout the shown training curve, while the validation curves become very close by later periods. This is consistent with the paper's observation that feature crossing can improve the linear model by representing relationships between prefix and type categories.

5. Part 2 - Additional Model: Random Forest Regressor

5.1 Model selection and justification

The reference paper uses linear regression, so the extension should add a different regression model rather than another implementation of the same model. Random Forest Regressor was selected because it belongs to a different, tree-based model family and can represent nonlinear relationships and interactions between categorical features. It is also practical for a dataset with only 500 examples and a modest tabular feature representation. The choice is therefore based on methodological contrast with the linear reference model, not on an assumption that the added model must outperform it.

Additional-model settings. The Random Forest uses 300 trees, random_state=42, max_features=1.0, min_samples_leaf=1, and all available processor threads. It is trained using the same 300-example training split and evaluated on the same 100-example validation and 100-example test splits. The feature representation uses the same prefix and type categories plus the prefix x type cross used by the stronger reference experiment.

Random Forest setting	Value
n_estimators	300

random_state	42
max_features	1.0
min_samples_leaf	1
Training / validation / test	300 / 100 / 100
Metric	RMSE

Split	Random Forest RMSE (ms)
Training	5382.47
Validation	5858.90
Testing	5881.74

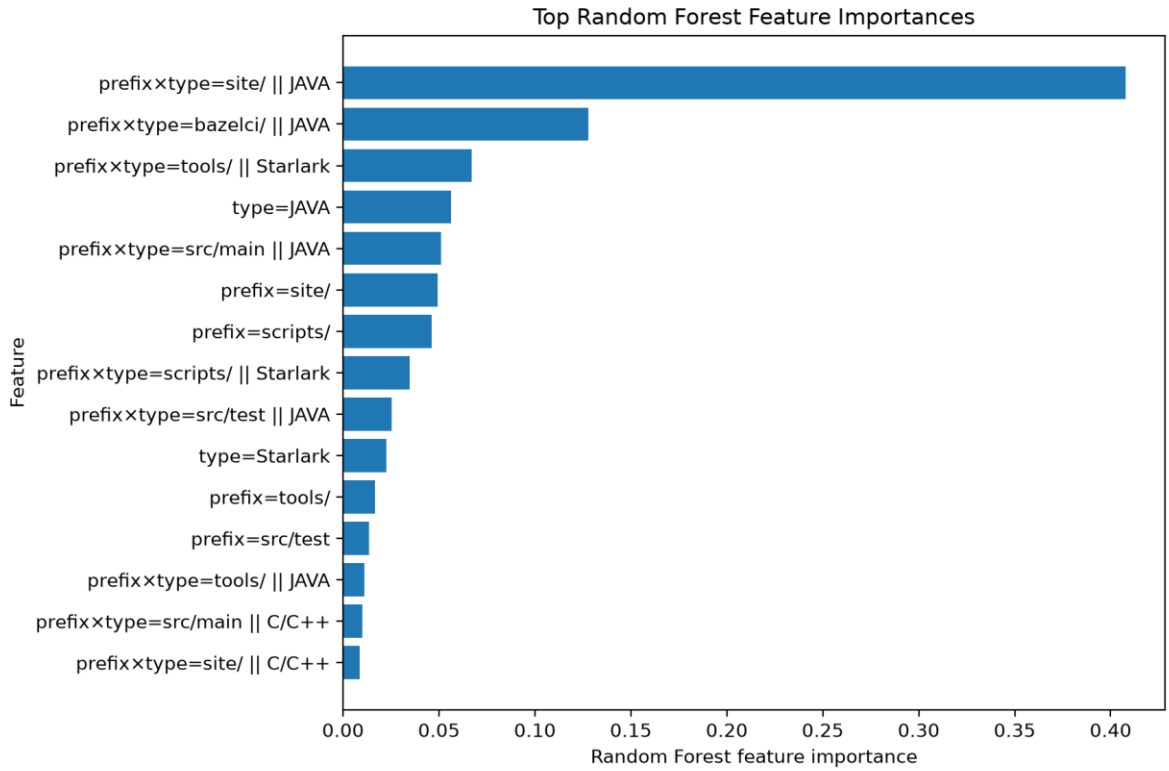


Figure 4. Top Random Forest feature importances for the extended model.

Feature importance note. The feature-importance plot indicates that several prefix x type combinations contribute strongly to the forest's predictions, with the site/JAVA combination the most important among the displayed features. These importances are model-specific measures of contribution to the Random Forest's split decisions and should not be interpreted as causal effects.

6. Comparative Analysis and Discussion

For Part 2, the fair comparison uses the final (period 9) predictions of the two reproduced linear-regression models on the same 100-example test split used by the Random Forest. This is distinct from the paper's

headline values, which are reported as average training RMSE values over the nine periods. Comparing the Random Forest test RMSE directly to the paper's 5735.84/5628.09 training averages would mix different datasets and evaluation stages, so that comparison is avoided.

Model	Final test RMSE (ms)	RF minus model (ms)	RF relative result
Linear Regression - without crossing	5796.58	+85.16	RF higher RMSE
Linear Regression - with prefix x type crossing	5801.74	+79.99	RF higher RMSE
Random Forest Regressor	5881.74	0.00	Reference point

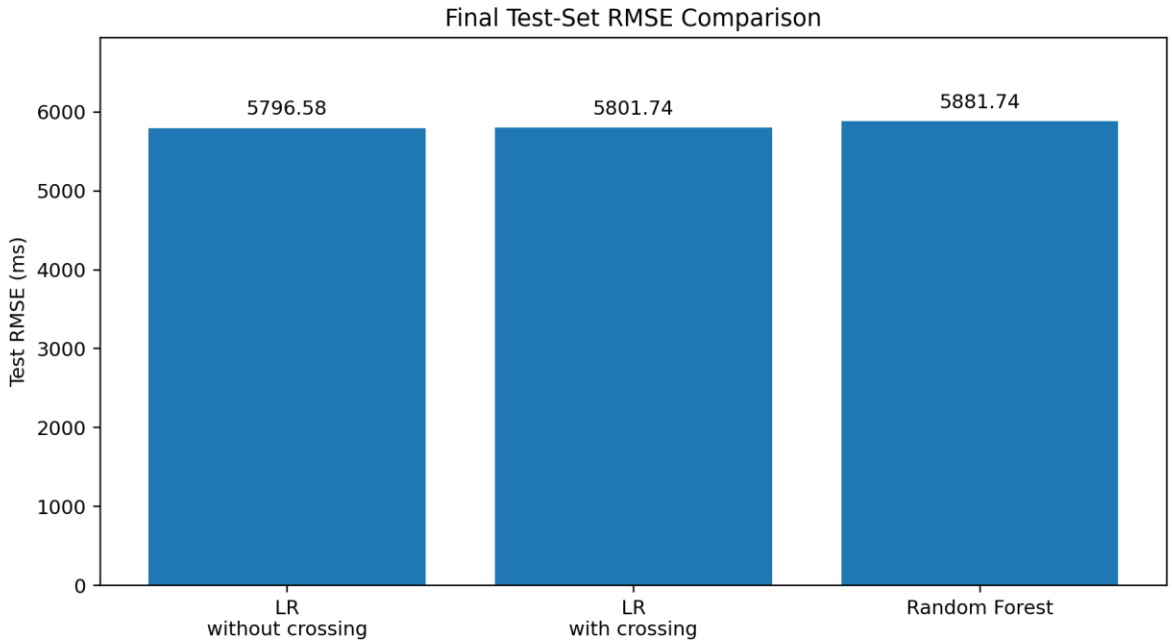


Figure 5. Final test-set RMSE comparison on the common 100-example test split.

Part 2 finding. Random Forest performed worse on the common test set. Its RMSE of 5881.74 ms is about 85.16 ms (1.47%) higher than the non-crossed linear model and about 79.99 ms (1.38%) higher than the feature-crossed linear model. Therefore, the additional model does not improve predictive RMSE in this particular reproduction.

Why this may have happened. The dataset is small, contains a highly concentrated file-type distribution, and uses a compact categorical feature space. The Random Forest also shows a train/test generalization gap: 5382.47 ms on training compared with 5881.74 ms on testing. That gap indicates that the forest fits the training data more closely than it fits unseen examples; the result is compatible with the possibility that the added flexibility did not generalize well on this small dataset. The outcome does not imply that Random Forest is intrinsically worse than linear regression for all datasets; it is a finding for this dataset, representation, and split.

Reference vs extension. The key outcome is that the extension requirement is satisfied even though the new model did not outperform the reference. The reference approach remains the lower-RMSE approach on the common test split in this experiment, while the Random Forest provides a useful nonlinear comparison model.

7. Instructor Requirement Traceability

Instructor requirement	How this project fulfills it
Read the reference and identify dataset, preprocessing, models, hyperparameters, metrics	Sections 2-4 identify the 500-sample dataset, categorical preprocessing, linear regression, feature crossing, learning rate 0.01, batch size 16, 9 periods, and RMSE.
Implement the same pipeline and models the reference uses	Part 1 independently reconstructs the categorical representation, linear-regression SGD training, and feature-crossing experiment from the paper and supplied datasets.
Use the same dataset/same metric	The supplied InputData.csv and CPUTimes.csv are used; RMSE is used throughout.
Compare Reported vs Your Result	Table in Section 4 compares the paper's 5735.84 and 5628.09 values against the reproduced 5707.02 and 5685.62 values.
Explain noticeable differences	Section 8 records the paper's unspecified split/shuffle details, missing grid candidates, and implementation-level differences as possible causes.
Build one additional model not in the reference	Part 2 adds a Random Forest Regressor.
Same dataset, split, and metric for extension	Random Forest uses the same 300/100/100 split, the same categorical/crossed representation, and RMSE.
Briefly justify model choice	Section 5 explains the choice as a nonlinear tree-based contrast to the paper's linear model.
Compare new model with reference and state better/worse/similar	Section 6 compares final test RMSE and states that Random Forest performed worse on the common test set.

8. Limitations and Reproducibility Notes

- The paper does not publish a random train/validation/test seed or a precise split procedure. The reproduction therefore uses the supplied row order for the documented 300/100/100 sizes.
- The paper states that a 3-fold grid search was used but does not list the candidate hyperparameter grid. The reproduction therefore uses the selected learning rate (0.01) and batch size (16) reported in the paper rather than inventing a candidate grid.
- The paper describes categorical feature conversion and feature crossing at a methodological level. The independent implementation uses fixed one-hot categorical blocks and a one-hot prefix x type cross, which is a transparent implementation of the stated categorical representation.
- The two supplied CSVs do not share a common commit identifier. The supplied CPUTime file is numbered 1 through 500, so rows are aligned by order. If an external version with an explicit shared identifier were available, that would remove this assumption.
- The reference paper's headline RMSE values are training RMSE averages over nine periods, whereas Part 2 requires a fair comparison on a common held-out test split. The report therefore keeps those two evaluation contexts separate.
- The reference paper itself notes that the dataset contains only 500 recent commits out of a much larger build history and that the input granularity is limited to common prefixes and file types. Those limitations also apply to this reproduction.

9. Conclusion

The project successfully addresses both parts of the mini-project specification. For Part 1, the documented Bazel build-time prediction pipeline was independently reconstructed from the reference paper and the supplied dataset. The reproduced average training RMSE values, 5707.02 ms without feature crossing and 5685.62 ms with prefix x type crossing, are close to the paper's reported 5735.84 ms and 5628.09 ms. The differences are small and can be explained by details that the paper does not fully specify, including the exact split/shuffle procedure and full hyperparameter grid.

For Part 2, a Random Forest Regressor was added as a distinct nonlinear model. On the same 100-example test split, it achieved 5881.74 ms RMSE, compared with 5796.58 ms for the reproduced linear model without crossing and 5801.74 ms for the feature-crossed linear model. Thus, Random Forest performed worse in this experiment. The result demonstrates that an additional model can be a valid extension even when it does not outperform the reference approach; the comparison remains useful because it tests whether greater nonlinear flexibility improves generalization on the supplied dataset.

Overall, the experiments reproduce the central finding of the reference that feature crossing can improve the linear-regression training result while also providing an independent extension experiment under the same evaluation conditions.

10. References

1. Muhammad Umar Nadeem and Sarah Raza, "Predicting Bazel Build Times Using Machine Learning," reference project report supplied for the mini project. The report describes the 500-commit Bazel dataset, categorical preprocessing, linear regression, feature crossing, and RMSE evaluation.
2. Muhammad Umar Nadeem, "CS229 Final Project," GitHub repository containing the publicly released project materials and processed dataset: https://github.com/umar-nadeem/CS229-Final-Project
3. Instructor-provided mini project guidelines, "Reproduce and Extend," Part 1 and Part 2 requirements supplied for this project.

Appendix A. Reproduction Artifacts

The implementation artifacts used to obtain the reported numbers are listed below so that the submission can be checked and rerun.

Artifact	Purpose
InputData.csv	500-row prefix/type input dataset
CPUTimes.csv	500-row CPUTime target dataset
part1_reproduction.py	Independent Part 1 implementation
part1_period_results.csv	Per-period train/validation/test RMSE values
part1_reported_vs_yours.csv	Reference reported values vs reproduced training values
part1_training_rmse.png	Training RMSE curve
part1_validation_rmse.png	Validation RMSE curve
part2_random_forest.py	Part 2 Random Forest implementation
part2_random_forest_results.csv	Random Forest train/validation/test RMSE

part2_model_comparison.csv	Final test-set comparison across models
part2_random_forest_test_predictions.csv	Per-sample Random Forest test predictions
part2_random_forest_feature_importance.png	Top Random Forest feature importances